

The background of the image features a dark blue gradient on the left, transitioning into a large, vibrant, abstract shape on the right. This shape is composed of overlapping curved segments in shades of orange, pink, and purple, creating a dynamic, modern aesthetic.

AWS re:Invent

NOV. 27 – DEC. 1, 2023 | LAS VEGAS, NV

D O P 3 1 4

Simplifying cloud infrastructure creation with the AWS CDK

Will Matos

Sr. SA Manager – DevOps & Mobile
Amazon Web Services

Andy Taylor

Sr. Specialist Solutions Architect,
Networking
Amazon Web Services



© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Who are we?

Will Matos

**Sr. SA Manager, DevOps & Mobile
Amazon Web Services**

Andy Taylor

**Sr. Specialist SA, Networking
Amazon Web Services**

Developer → infrastructure

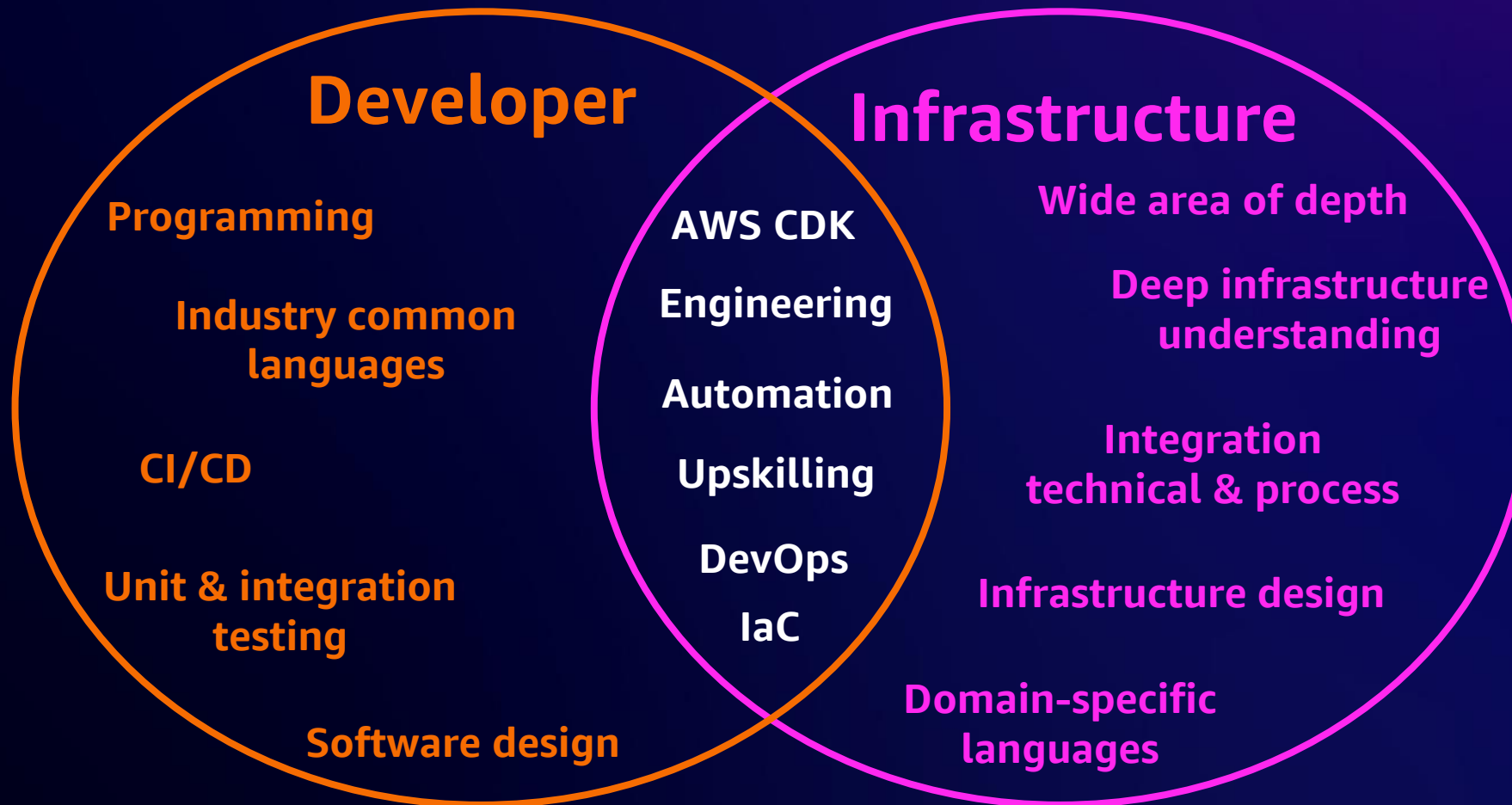
- 25 years of technology experience
- Software development, IT management, solutions architecture
- Global Leader, Specialist Solutions Architects
- Next-generation developer experience

Infrastructure → developer

- 30 years in networking (mostly data center for large enterprise)
- AWS Networking Specialist Solutions Architect
- Developed a passion (verging on obsession) for networking as code/infrastructure as code
- Learning to become a proficient developer and loving it

Who are you?

🕶️ Time to be interactive 🕶️



Why the AWS CDK?



84 lines of code

18 ./bin/vpc.ts
66 ./lib/vpc-stack.ts

1,292 lines
JSON template

NOTE: This is for **ONE** VPC

From

To

10x VPCs

122 lines of code

40 ./bin/vpc.ts
72 ./lib/vpc-stack.ts

13,761 lines
over 10 JSON templates

```
import * as cdk from 'aws-cdk-lib';
import * as ec2 from 'aws-cdk-lib/aws-ec2';
import * as iam from 'aws-cdk-lib/aws-iam';
import { Construct } from 'constructs';

export class VpcStack extends cdk.Stack {
  constructor(scope: Construct, id: string, props?: cdk.StackProps) {
    super(scope, id, props);

    // Number of AZ for the VPC
    const maxAZs = 2;

    // Create the VPC with 3 subnets
    const vpc = new ec2.Vpc(this, 'Vpc', {
      subnetConfiguration: [
        {
          name: 'public',
          subnetType: ec2.SubnetType.PUBLIC,
        },
        {
          name: 'private-with-nat',
          subnetType: ec2.SubnetType.PRIVATE_WITH_EGRESS,
        },
        {
          name: 'private',
          subnetType: ec2.SubnetType.PRIVATE_ISOLATED,
        },
      ],
      maxAZs: maxAZs,
      natGateways: maxAZs,
    });

    // Add SSM VPC endpoints to the private subnets
    vpc.addInterfaceEndpoint('ssm-messages', {
      privateDnsEnabled: true,
      service: ec2.InterfaceVpcEndpointAwsService.SSM_MESSAGES,
      subnets: vpc.selectSubnets({
        subnetType: ec2.SubnetType.PRIVATE_ISOLATED,
      }),
    });

    vpc.addInterfaceEndpoint('ec2-messages', {
      privateDnsEnabled: true,
      service: ec2.InterfaceVpcEndpointAwsService.EC2_MESSAGES,
      subnets: vpc.selectSubnets({
        subnetType: ec2.SubnetType.PRIVATE_ISOLATED,
      }),
    });

    vpc.addInterfaceEndpoint('ssm', {
      privateDnsEnabled: true,
      service: ec2.InterfaceVpcEndpointAwsService.SSM,
      subnets: vpc.selectSubnets({
        subnetType: ec2.SubnetType.PRIVATE_ISOLATED,
      }),
    });

    // Instance role that allows SSM to connect
    const role = new iam.Role(this,
      'InstanceRoleWithSSMPolicy', {
        assumedBy: new iam.ServicePrincipal('ec2.amazonaws.com'),
      });
    role.addManagedPolicy(
      iam.ManagedPolicy.fromAwsManagedPolicyName(
        'AmazonSSMManagedInstanceCore'
      )
    );
  }
}
```

Now imagine 1,000 or 5,000 VPCs!

Capitalize on a common programming language

A multi-language software development framework for modeling cloud infrastructure as reusable components

```
class UrlShortener extends Stack {  
  constructor(scope: App, id: string, props?: UrlShortenerProps) {  
    super(scope, id, props);  
  
    const vpc = new ec2.Vpc(this, 'vpc', { maxAzs: 2 });  
    const cluster = new ecs.Cluster(this, 'cluster', { vpc: vpc });  
    const service = new patterns.NetworkLoadBalancedFargateService(this, 'sample-app', {  
      cluster,  
      taskImageOptions: {  
        image: ecs.ContainerImage.fromAsset('ping'),  
      },  
      dom  
    });  
    // Setup AutoScaling policy  
    const scaling = service.service.autoScaleTaskDefinition(scaling.scaleOnCpuUtilization('CpuScaling',  
      targetUtilizationPercent: 50,  
      scaleInCooldown: Duration.seconds(60),  
      scaleOutCooldown: Duration.seconds(60)  
    ));  
  }  
}
```

domainName	(property) patterns.NetworkLoadBalancedServiceBaseProps.domainName?: string undefined
domainZone	The domain name for the service, e.g. "api.example.com."
	@default
	- No domain name.



Familiar
Your language



Tool support
Autocomplete
inline documentation



Abstraction
Sane defaults
Reusable classes



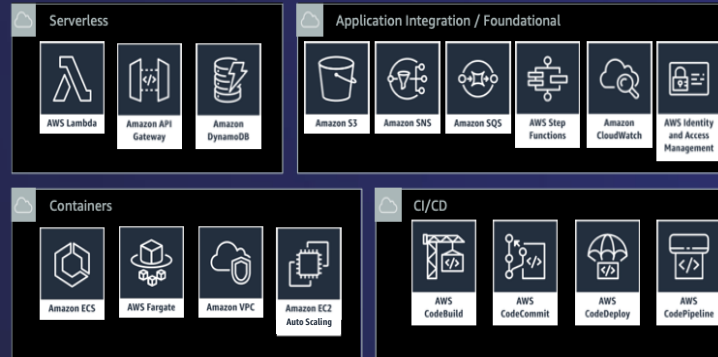
AWS Cloud Development Kit (AWS CDK)

Main components

App

Stacks

Resources



```
~/hello-cdk (zsh)
> cdk diff hello-cdk-1
Stack hello-cdk-1
Resources
[-] AWS::SNS::Topic MyFirstTopic MyFirstTopic0ED1F8A4
  [-] DisplayName
    [-] My First Topic Yeah
    [+ Hello, CDK!

~/hello-cdk master* 6s
> cdk deploy hello-cdk-1

hello-cdk-1: deploying...
hello-cdk-1: creating CloudFormation changeset...
0/3 | 12:25:30 PM | UPDATE_IN_PROGRESS | AWS::CDK::Metadata | CDKMetadata
0/3 | 12:25:30 PM | UPDATE_IN_PROGRESS | AWS::SNS::Topic | MyFirstTopic (MyFirstTopic0ED1F8A4)
1/3 | 12:25:31 PM | UPDATE_COMPLETE | AWS::SNS::Topic | MyFirstTopic (MyFirstTopic0ED1F8A4)
2/3 | 12:25:32 PM | UPDATE_COMPLETE | AWS::CDK::Metadata | CDKMetadata
2/3 | 12:25:35 PM | UPDATE_COMPLETE_CLEANUP | AWS::CloudFormation::Stack | hello-cdk-1
3/3 | 12:25:36 PM | UPDATE_COMPLETE | AWS::CloudFormation::Stack | hello-cdk-1

[✓] hello-cdk-1

Stack ARN:
arn:aws:cloudformation:us-east-1:585695036304:stack/hello-cdk-1/b9da27f0-fafe-11e9-b7b3-12f2c8f206e2

~/hello-cdk master* 33s
>
```

Core Framework

AWS Construct Library

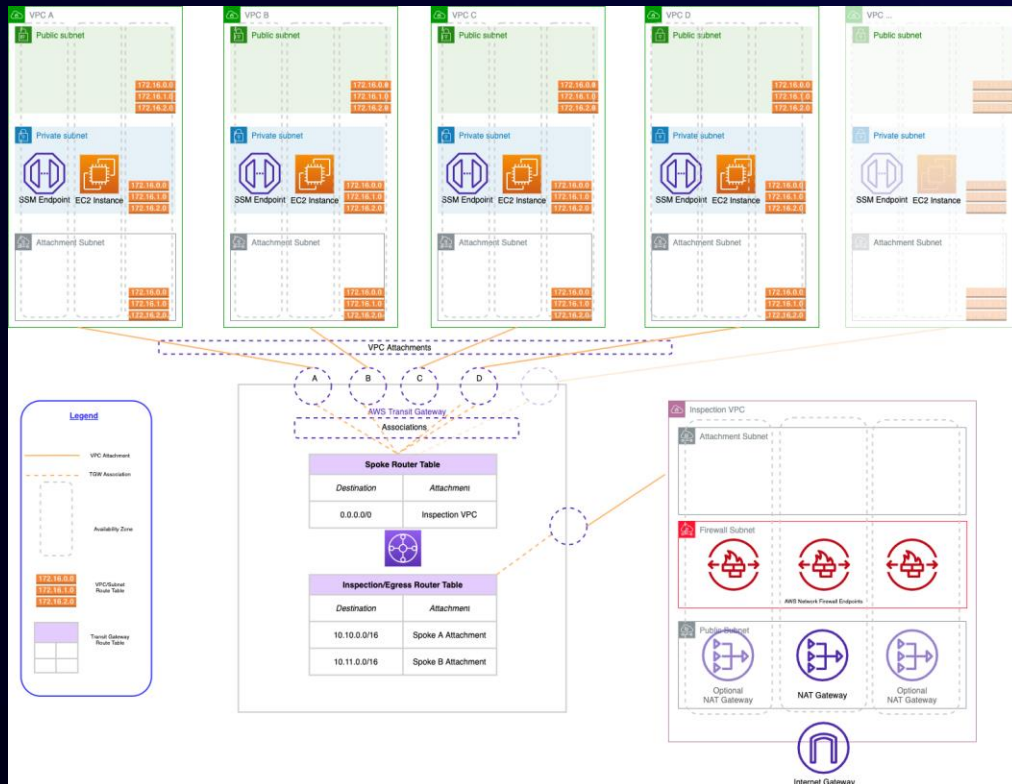
AWS CDK CLI

An example repo used in production

<https://github.com/aws-samples/hub-and-spoke-with-inspection-vpc-cdk>

Source: Golang, Python, and TypeScript

Target architecture



CDK Pipelines

The code in the sample repo creates:

- **Continuous deployment** (CD) pipeline
- Self-mutating to provision a **scalable** number of VPCs
- **Governance and audit**
- **Unit** and **integration** testing
- Separation of duties
 - Networks – network team
 - Firewall policy – firewall/security team

Construct levels

L3+

Purpose-built constructs

Opinionated abstractions;
also known as patterns

L2

AWS constructs

High-level service constructs

L1

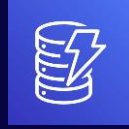
CloudFormation resources

Automatically generated

Custom construct libraries

AWS

L2 constructs



Amazon DynamoDB



Amazon RDS



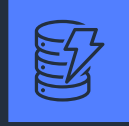
Amazon EC2



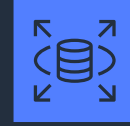
AWS Lambda

L3 constructs

Organization defaults



Your DynamoDB



Your RDS



Your EC2

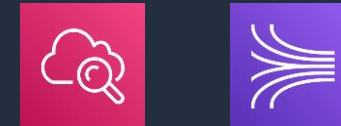


Your Lambda

Common use cases

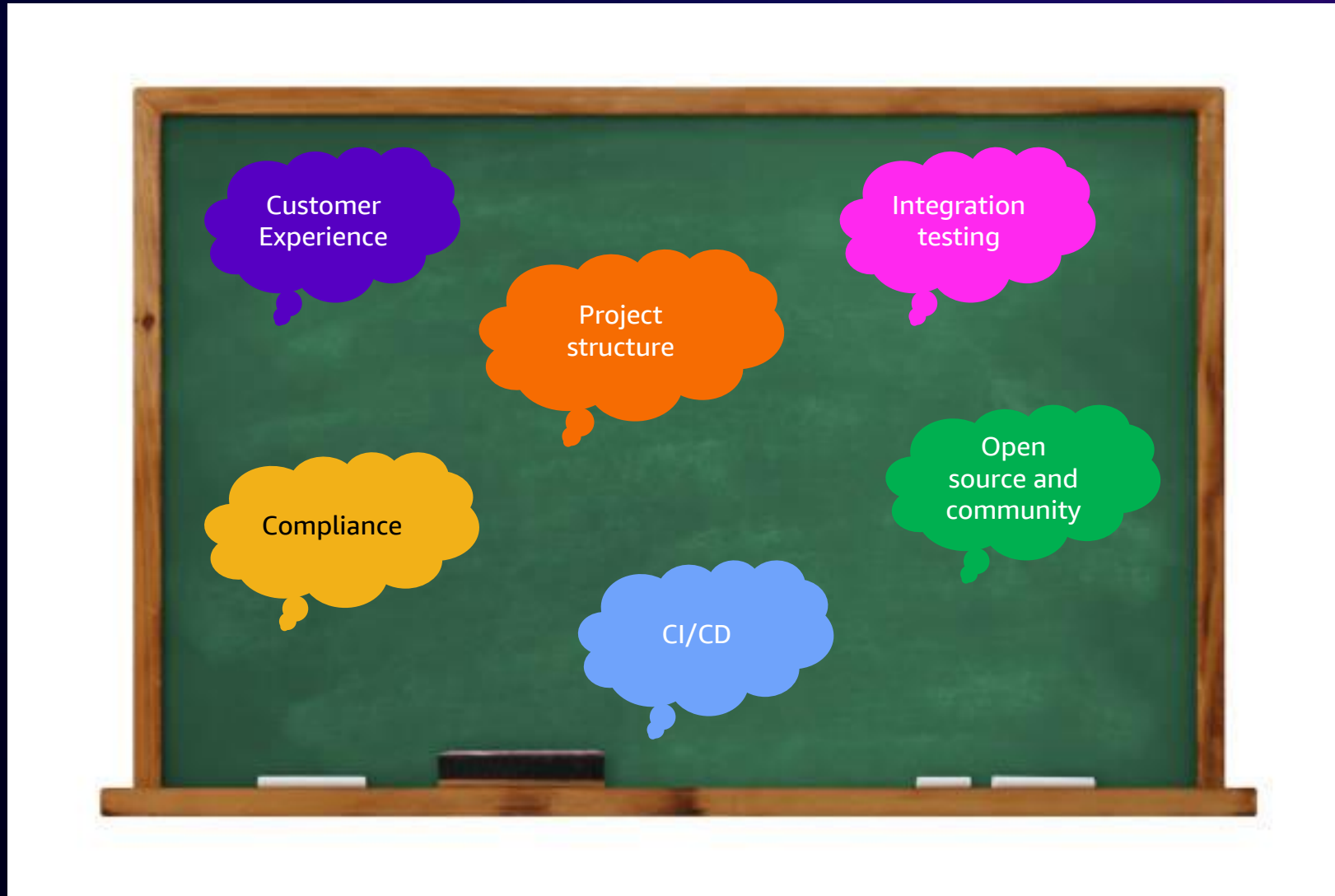


Secure REST API



Amazon CloudWatch
to Amazon Kinesis

Chalk talk (we can take complex items offline)



Thank you!

Will Matos

wilmatos@amazon.com

Andy Taylor

taylaand@amazon.co.uk



Please complete the session
survey in the mobile app

